Gestão de Projecto de Software

Licenciatura em Engenharia Informática

2024/2025

Pre-Game (ou Sprint 0)

A6

199

199

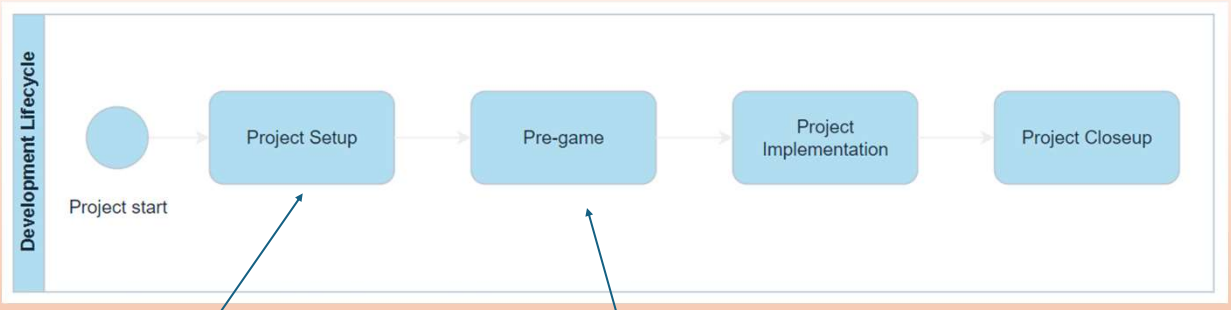
Actividade Mentimeter

GPS - 2024_2025 m1 Pre-Game

200

200

Development Lifecycle

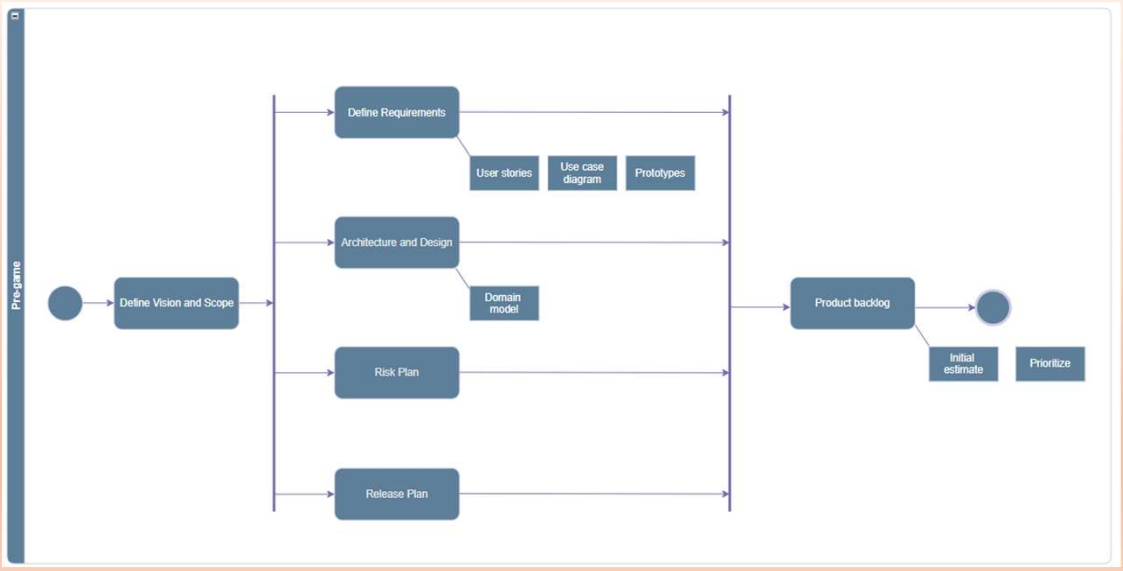


São criadas as bases do projecto:

- a criação de equipas,
- a definição de ferramentas de gestão e comunicação,
- o estabelecimento de infraestruturas técnicas, como repositórios de código e ambientes de desenvolvimento.

o pré-game pode ser visto como uma fase de preparação estratégica para garantir que, quando a equipa entra no jogo (no sprint), estão prontos para trabalhar com um objectivo claro e com as ferramentas e processos necessários para a sua execução eficiente

Pre-Game (or Sprint 0)



README.md document

1. Team
2. Vision and Scope
3. Requirements
   - Use case diagram
   - Mockups
   - User Stories (links to GitLab issues)
4. DoD (Definition of Done)
1. Architecture and Design
2. Risk Plan
3. Pre-Game
4. Release Plan
5. Increments
   - Sprint 1
   - …

Team

- Team members:
  - Name1, student number, GitLab email
  - Name2, student number, GitLab email
  - Name3, student number, GitLab email
  - …

©2011-2023 João Cunha, DEIS-ISEC

Vision and Scope

1 Problem Statement

- 1.1 Project background
- 1.2 Stakeholders
- 1.3 Users

2 Vision & Scope of the Solution

- 2.1 Vision statement
- 2.2 List of features
- 2.3 Features that will not be developed
- 2.4 Assumptions

DoD (Definition of Done)

- It is a collection of criteria that must be completed for a User Story to be considered "done."
- Example:
  1. All tasks done
     1. CI – built, tested (JUnit), reviewed (SonarCloud)
     2. Merge request to **qa** (code review)
  2. Acceptance tests passed
  3. Accepted by the client
  4. Code merged to **main**

Risk Plan

- Threshold of Success:
  1. By the 2nd release date, 30% of all the User Stories from the Product Backlog are closed.
  2. *Goal2 …*
- Risk List
  - RSK2 – PxI: 4x5=20; The team has no experience in software estimation, **therefore** there might cause underestimation, delaying the project in an uncontrolled way
  - *RSKn – PxI: axb=c; Risk statement*
  - …
- Mitigation Actions (threats>=20)
  - RSK2 – MS; At the middle of each sprint, compare the progress of the project with the estimation and, if necessary, review the sprint plan.
  - *RSKn – CP/AS/MS; Contingency Plan (CP), Avoidance Strategy (AS) or Minimization Strategy (MS)*
  - …

Pre-Game

- Sprint 0 Plan
  - Goal: description
  - Dates: XXXXXXX , 2 weeks
  - Sprint 0 Backlog:
    - Task1 – Write Team
    - Task2 – Write V&S
    - Task3 – Write Requirements
    - Task4 – Write DoD
    - Task5 – Write Architecture&Design
    - Task6 – Write Risk Plan
    - Task7 – Write Pre-Game
    - Task8 – Write Release Plan
    - Task9 – Write Product Increments
    - Task10 – Create Product Board
    - Task11 – Create Sprint 0 Board
    - Task12 – Write US in PB, estimate (SML), prioritize (MoSCoW), sort
    - Task13 – Create repository with “GPS Git” Workflow

Release Plan

permite à equipa planear e comunicar o que será entregue em cada versão do produto. objectivos, datas e versões, ajudam a equipa a organizar-se e a alinhar expectativas com os stakeholders.
As iterações de versões (V1.0, V2.0) também permitem rastrear o progresso e as melhorias do produto ao longo do tempo.

- Release 1
  - Goal: MVP – description
  - Date:
  - Release: V1.0
- Release 2
  - Goal: Final release – description
  - Date:
  - Release: V2.0

Minimum **V**iable **P**roduct (Produto Mínimo Viável).
O objectivo é entregar uma funcionalidade básica que satisfaça as necessidades essenciais dos utilizadores.
O MVP é projectado para validar as hipóteses de negócio com o menor investimento possível, permitindo que a equipa receba feedback real dos utilizadores. Descrição: Aqui, deve-se descrever as principais funcionalidades que estarão incluídas no MVP. Por exemplo, se for uma aplicação de e-commerce, o MVP pode incluir funcionalidades como a criação de contas de utilizador, pesquisa de produtos e finalização de compras.

O uso de versões permite que a equipa e os stakeholders acompanhem as alterações e melhorias feitas ao longo do tempo

A notação V2.0 indica que esta é uma versão significativa em relação à V1.0,

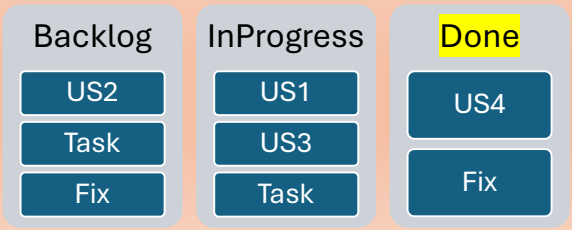
O objectivo da segunda versão é alcançar um lançamento final. Isso implica que o produto está agora mais completo e contém todas as funcionalidades planeadas que foram desenvolvidas e testadas com base no feedback recebido após o lançamento do MVP.

Increments

- Sprint 1
  - Sprint Plan
    - Goal: *what's the goal for this sprint*
    - Dates: XXXXXX| *3* weeks
    - Roles:
      - PO: *name*
      - SM: *name*
    - To do:
      - *(list of US or Tasks from the PB, with links)*
      - US1: *As … I want … so that …*
      - Task1: *Some task*
    - Story Points: *2*S+*3*M+*3*X+*2*H
    - Analysis: *short analysis of the planning meeting*
  - Sprint Review
    - Analysis: *what was not done or what was added (Link to US or Task from the PB)*
    - Story Points: *2*S+*1*M+*2*X+*2*H
    - Version: 0.1
    - Client analysis: *client feedback*
    - Conclusions: *what to add/review*
  - Sprint Retrospective
    - What went well:
    - What went not well:
    - What to improve:
- Sprint 2
  - …

GPS Product Backlog (in GitLab)

- Product backlog items (or cards) represented as **issues**, in an **issue board**
  - Note: When creating the "Issue Board" in GitLab, remove default "Open" and "Closed" lists
- **Issues board** configuration:
  - **Backlog** (where the product backlog cards are placed)
    - US[X] – User Story number X
  - **InProgress** (when the corresponding cards are copied to the Sprint Backlog)
    - When a US[X] is InProgress, the corresponding AT[X] and Tasks are created in the Sprint Backlog
  - ~~Closed~~ **Done** (when the corresponding cards are ~~Closed~~ Done in the Sprint Backlog)
    - When an AT[X] is ~~Closed~~ Done in the Sprint Backlog, the US[X] is also ~~Closed~~ Done



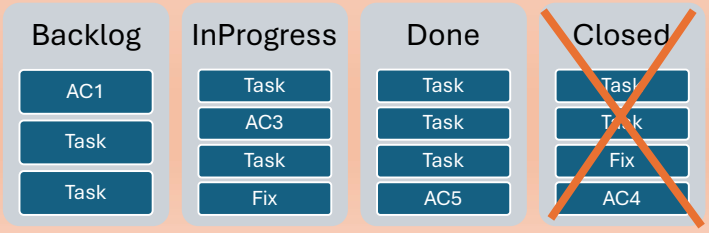
Labels for the Product Backlog

List labels	Card Types	Size Estimation (for US cards only)	Priority (for US cards only)
Product Backlog BacklogPB	User Story <number> US1	Large L	Must have Must
In Progress InProgressPB	Configuration task conf	Medium M	Should have Should
Done DonePB	Documentation task doc	Small S	Can have Can
	Management task man		Won't have Won't
	Any fix fix		

- GPS Product Backlog (issue board) has 3 lists (see **List Labels**)
- The cards (issues) are:
  - US (numbered)
  - 3 "task" types
  - 1 "fix", of any type
- The US cards may have:
  - One of three size estimations
  - One of four priorities

GPS Sprint<x> Backlog (in GitLab)

- Sprint backlog cards represented as **issues**, in an **issue board**
- Each Sprint has its own backlog (issue board)
  - Note: When creating the "Issue Board" in GitLab, remove default "Open" and "Closed" lists
- **Issues board** configuration:
  - **Backlog** (where the sprint backlog cards are placed)
    - Each User Story will have its corresponding Acceptance Criteria card (e.g. AC1)
  - **InProgress** (when the cards are being developed; if code, in **dev** branch)
  - **Done** (when the cards are being validated; or the code is in **qa** branch)
  - ~~**Closed** (when the cards are "done"; or the if code is in **main** branch)~~



213

Labels for the Sprint Backlog

List labels	Card Types	Related US (for USTask and AC cards only)
Sprint Backlog BacklogS1	US Task USTask	User Story <number> US1
In Progress InProgressS1	Acceptance Criteria AC	
Done DoneS1	Configuration task conf	
~~Closed~~ ClosedS1	Documentation task doc	
	Management task man	
	Any fix fix	

- GPS Sprint Backlog has ~~4~~ 3 lists
- The cards (issues) are:
  - **USTask**
  - **AC**
  - 3 "**task**" types
  - 1 "**fix**", of any type
- All card types should have an Estimated time
- The U ask and AC cards should have:
  - A related User Story

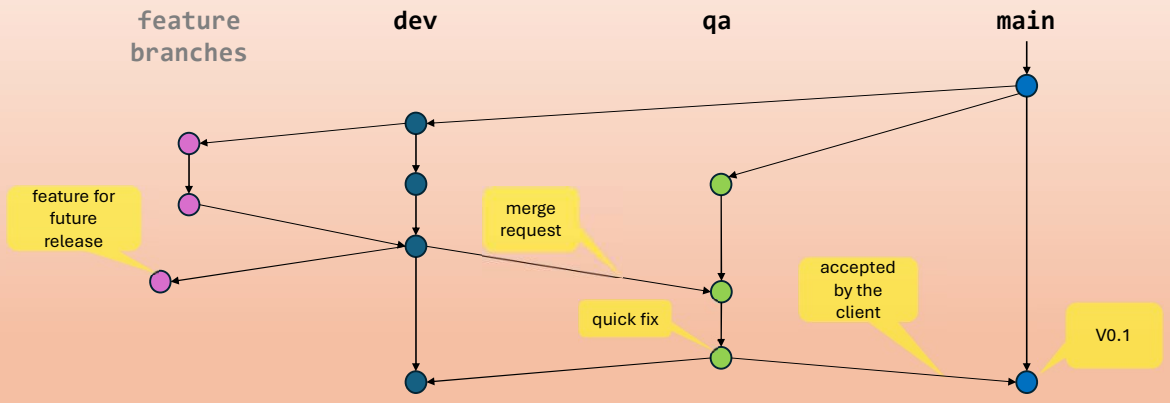
214

Backlog card types

card	Title	Description	Estimate	Priority	Tasks	Labels	Boards
User Story	US[X]: As a ..., I want ... so that...	Short description; Link to acceptance tests	S, M, L story points	MoSCoW	None	US[X]	PB
User Story Task	USTask: Short description	Full description of the task	0, 1, 2, 3, 5 or 8 hours		necessary subtasks	US[X] & USTask	SB
Task – documentation, configuration, management	Task: Short description	Full description	0, 1, 2, 3, 5 or 8 hours		necessary subtasks	doc | conf | man	PB or SB
Fix – code, documentation, configuration	Fix: Short description	Full description	0, 1, 2, 3, 5 or 8 hours		necessary subtasks	Fix & (US[X] | doc | conf)	SB or PB
Acceptance Criteria/Test	AC: Acceptance criteria for US[X]	All acceptance criteria/tests for the US[X]	0, 1, 2, 3, 5 or 8 hours		one per acceptance test plus DoD criteria	US[X] & AC	SB

GPS Git workflow

- Branches





Planning

Release Planning

- High-level document that outlines the scope, timeline, and goals of the releases of a product. The first is usually an MVP (Minimum Viable Product)
- Not part of Scrum framework, but is often used
- It's necessary to have:
  - What the Client needs/wants, and when
  - Risk plan, with mitigation tasks
  - Most architectural relevant parts
  - Release time
  - A list of User Stories (in the **Product Backlog**):
    - Priotitized (ex: MoSCoW)
    - Estimated (in Story Points, ex: XS, S, M, L, XL)
  - Team capacity (hours/week * team members * duration in weeks)

Release Planning

- How to plan:
  - Order the US, by putting on top:
    - Those with the highest business value (chosen by the client)
    - The most architectural relevant
    - Those with the highest risk
    - Those with the highest priority
  - Select some related US from the top of the list. Define a goal, that represents an MVP
  - Have a rough idea of the effort to develop the MVP: convert story points into hours (ex: S:0-2; M:3-5; L:6-10; XL:11-18)
  - Compare with the Team Capacity for the release date. Adjust as necessary.
    - Team Capacity = week hours per person * number of weeks * number of persons
  - Write the Release goal.
- Continue for the next releases

Sprint Planning

1. Based on the next Release plan and top of the Product Backlog, select the related User Stories for the next sprint
2. Estimate the Team Capacity for the sprint. Check if the US can be finished.
3. Extract the Tasks and Acceptance Criteria for each US, and add them to the **Sprint Backlog**
4. Estimate the effort for each Task and AC (in hours)
5. Verify if the Tasks and AC can be done in the sprint. Review if necessary
6. Write the Sprint Goal


isec
Politécnico de Coimbra
Gestão de Projecto de Software
Licenciatura em Engenharia Informática
2024/2025
A7
221

221